

CENTRO UNIVERSITÁRIO ÁLVARES PENTEADO - FECAP

CIÊNCIA DA COMPUTAÇÃO

Engenharia de Software e Arquitetura de Sistemas

APLICAÇÃO DE DESIGN DE SOFTWARE E DIAGRAMAS UML NO PI

Dashboard de Indicadores Cannoli - TechSnack DevTeam

Integrantes:

Esther Oliveira Costa

Higor Luiz Fonseca dos Santos

João Victor de Faria Santana

Professores orientadores: Eduardo Savino Gomes, Lucy Mari Tabuti, Maurício Lopes da Cunha, Rodnil da Silva Moreira Lisboa e Victor Bruno Alexander Rosetti de Quiroz

SÃO PAULO
2026

Sumário

1. Introdução
2. Contexto do projeto
3. Aplicação de design de software
4. Diagramas UML aplicados ao projeto
5. Resultados aplicados no projeto: entradas e saídas
6. Nossas observações reflexivas
7. Conclusão

1. Introdução

Neste documento, descrevemos como aplicamos design de software e diagramas UML no Projeto Integrador desenvolvido pela TechSnack DevTeam. Nosso projeto, denominado Dashboard de Indicadores Cannoli, teve como objetivo centralizar dados operacionais e comerciais da Cannoli em uma plataforma web capaz de transformar informações dispersas em indicadores visuais, filtros, recomendações e apoio à tomada de decisão.

Estruturamos a documentação com base na arquitetura que implementamos durante o desenvolvimento: frontend em React/Vite, backend em Node.js/Express, banco de dados MySQL, processamento analítico em Python, integração com API de e-mail transacional e hospedagem em Vercel, Render e Railway.

Nosso objetivo neste relatório é demonstrar como nossas decisões de design de software, separação de responsabilidades e modelagem UML nos ajudaram a organizar o sistema, reduzir acoplamentos, melhorar a rastreabilidade das funcionalidades e identificar pontos críticos de desempenho e integração.

2. Contexto do projeto

Identificamos que a Cannoli precisava de uma solução capaz de consolidar dados de empresas, clientes, pedidos, campanhas, métricas financeiras e recomendações estratégicas em uma interface única. O problema central era transformar dados operacionais em informações úteis, acessíveis e compreensíveis para usuários administrativos e empresas parceiras.

Como solução, desenvolvemos um dashboard web com controle de acesso por perfil. No painel global, administradores e colaboradores Cannoli consultam KPIs, gráficos, filtros, recomendações, clientes, empresas, campanhas, importação de dados, configurações e convites. Na visão por empresa, restringimos os dados ao contexto da empresa autenticada, garantindo separação lógica das informações.

Camada	Tecnologia principal	Responsabilidade no sistema
Frontend	React + Vite	Interface, rotas, formulários, dashboards, filtros e visualização de dados
Backend	Node.js + Express	APIs REST, autenticação, regras de negócio, convites, recuperação de senha e integração com serviços externos
Banco de dados	MySQL / Railway	Persistência de usuários, empresas, convites, tokens, campanhas, pedidos e configurações
Processamento analítico	Python	Cálculo de KPIs, RFM, rankings, alertas, recomendações e geração do JSON analítico
E-mail transacional	Brevo API	Envio de códigos de recuperação de senha e convites
Deploy	Vercel, Render e Railway	Publicação do frontend, backend e banco de dados em ambiente hospedado

3. Aplicação de design de software

3.1 Separação em camadas

Organizamos o sistema em camadas para evitar que regras de negócio, persistência, rotas HTTP e interface ficassem misturadas. No backend, trabalhamos com controllers, services, routes, middlewares, configurações de banco e utilitários. No frontend, separamos páginas, componentes, serviços de API,

constantes, formatadores e seções do dashboard. Essa divisão nos permitiu alterar partes específicas do sistema sem comprometer todo o projeto.

Camada de design	Aplicação prática no projeto	Benefício
Routes	Mapeamento das URLs da API, como /api/auth, /api/admin-dashboard e /api/staff-invites	Centraliza os pontos de entrada HTTP
Controllers	Recebem requisições, validam dados básicos e retornam respostas padronizadas	Reduz duplicidade e melhora a leitura do fluxo
Services	Executam regras de negócio e consultas ao banco	Evita que SQL e regras fiquem diretamente nas rotas
Middlewares	Autenticação JWT, autorização por perfil e CORS	Controla segurança e acesso de forma reutilizável
Utils	Envio de e-mail, formatação e funções auxiliares	Isola integrações externas e simplifica manutenção
Python analytics	Processamento dos dados brutos e geração de indicadores	Separa cálculo analítico da API web

3.2 Padrões e decisões de arquitetura

Aplicamos um design próximo ao padrão MVC adaptado para APIs REST. As rotas representam os endpoints, os controllers coordenam as requisições, os services concentram as regras de negócio e o banco de dados persiste as entidades principais. No frontend, estruturamos componentes reutilizáveis para cards de KPI, gráficos, badges de status, filtros e seções administrativas.

Também adotamos decisões de design orientadas à confiabilidade: usamos variáveis de ambiente para separar dados sensíveis do código, JWT para autenticação, logs de performance para medir endpoints lentos, cache no processamento do dashboard, CORS configurado para os ambientes hospedados e envio de e-mails por API HTTP da Brevo para evitar timeouts de SMTP no Render.

Decisão de design	Problema tratado	Resultado aplicado
Autenticação por JWT	Controlar acesso a rotas protegidas	Usuário autenticado recebe token e acessa rotas conforme seu perfil
Controle por perfil	Separar admin, colaborador e empresa	Menus, filtros e dados são apresentados conforme permissões
Envio assíncrono de e-mail	Rotas travavam esperando SMTP	Convites e recuperação respondem rapidamente, com e-mail processado separadamente
Brevo API em vez de SMTP	Timeout em hospedagem gratuita	Comunicação por HTTP/HTTPS mais estável para e-mails transacionais
Logs [PERF]	Dificuldade de identificar lentidão	Cada endpoint passou a registrar tempo de resposta
Cache no dashboard	Processamento Python repetido	Redução de execuções desnecessárias em consultas semelhantes

4. Diagramas UML aplicados ao projeto

4.1 Diagrama de casos de uso

No diagrama de casos de uso, representamos as principais interações entre os perfis de usuário e a plataforma. Com ele, evidenciamos que nossa solução não é apenas um dashboard visual, mas um ambiente administrativo com autenticação, convites, recuperação de senha, importação de dados, filtros analíticos e visualizações específicas por perfil.



Figura 1 - Diagrama de casos de uso do Dashboard de Indicadores Cannoli.

4.2 Diagrama de classes

No diagrama de classes, representamos as principais entidades persistidas e processadas no sistema. Incluímos usuários, empresas, convites, tokens de recuperação de senha, pedidos, campanhas e o objeto consolidado do dashboard. Essa modelagem nos ajudou a compreender as relações entre autenticação, perfis de acesso, dados comerciais e processamento analítico.

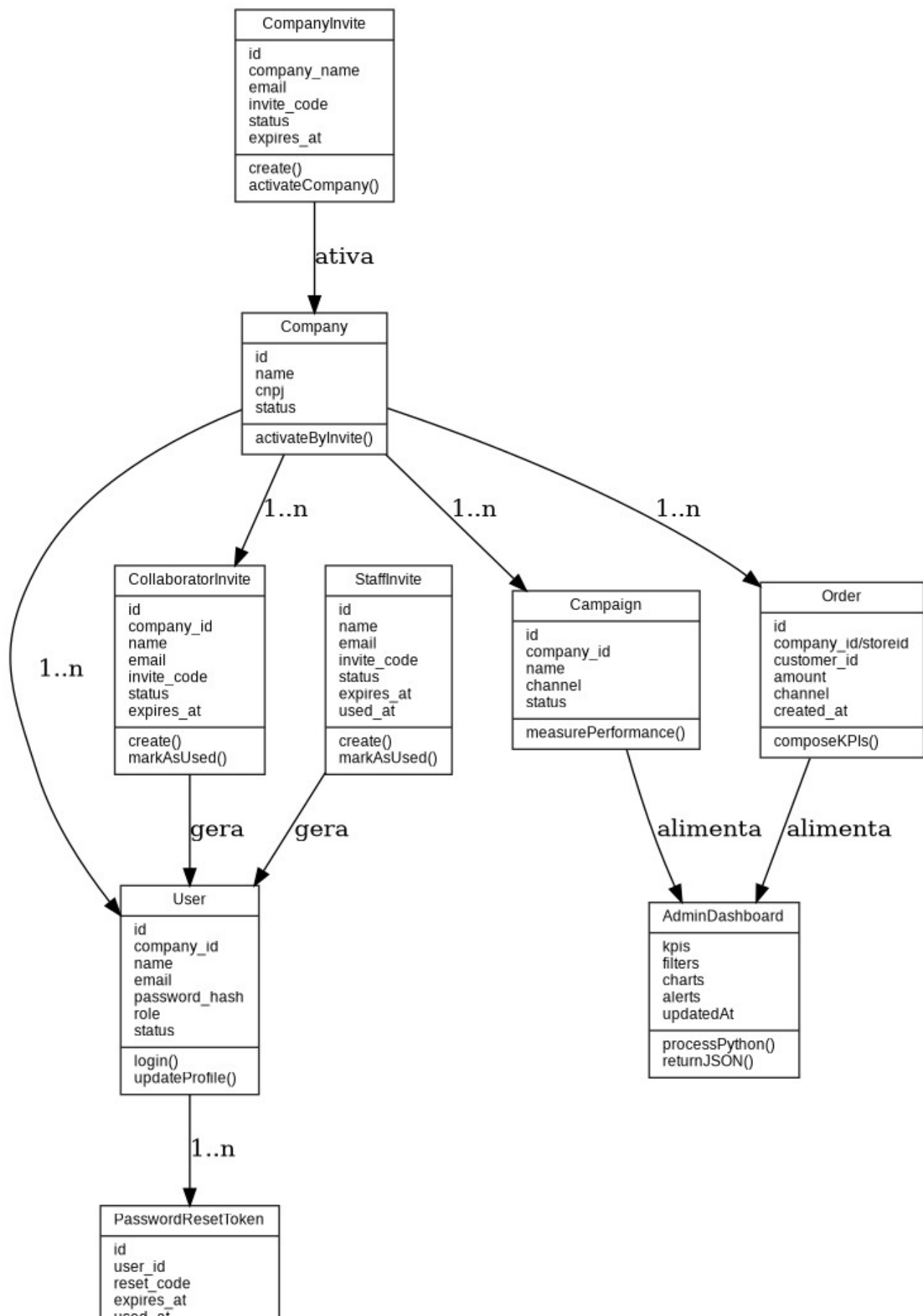


Figura 2 - Diagrama de classes simplificado das entidades centrais do projeto.

4.3 Diagrama de sequência do dashboard

No diagrama de sequência, demonstramos o fluxo que implementamos quando o usuário acessa o painel global e utiliza filtros. A requisição sai do frontend com o token JWT, passa pelo backend, aciona o processamento analítico quando necessário, consulta ou atualiza o arquivo consolidado do dashboard e retorna os dados para renderização dos KPIs e gráficos.

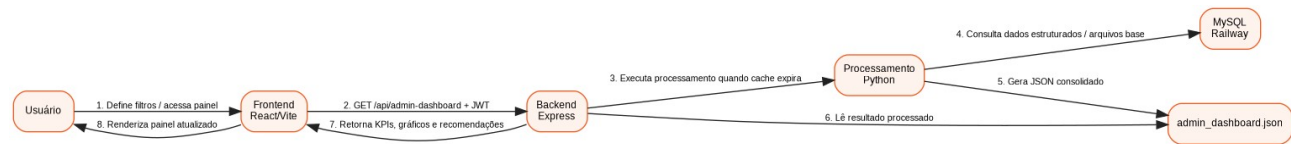


Figura 3 - Diagrama de sequência do fluxo de consulta e processamento do dashboard.

5. Resultados aplicados no projeto: entradas e saídas

A aplicação dos conceitos de design de software resultou em fluxos com entradas e saídas bem definidas. Essa organização foi essencial para validarmos o comportamento do sistema, depurarmos erros e demonstrarmos que cada módulo possui responsabilidades claras.

Funcionalidade	Entrada	Processamento	Saída
Login	E-mail e senha do usuário	Validação no MySQL, comparação de senha criptografada e geração de JWT	Token, dados do usuário e redirecionamento para o painel correto
Recuperação de senha	E-mail cadastrado	Geração de código/token e envio por Brevo API	Mensagem ao usuário e código enviado por e-mail
Convite de colaborador Cannoli	Nome e e-mail do colaborador	Criação de convite no banco e disparo de e-mail transacional	Convite ativo com código de cadastro
Convite de empresa	Dados da empresa e e-mail responsável	Geração de código de ativação de empresa	E-mail com código de ativação e registro pendente/ativo
Dashboard global	Filtros de período, empresa, canal e tipo de pedido	Processamento em Python e consolidação em JSON	KPIs, gráficos, rankings, alertas e recomendações
Abas analíticas	Filtros reutilizados por página	Recalculo ou reaproveitamento dos dados do dashboard	Visões de empresas, campanhas, clientes, financeiro e recomendações
Importação de dados	Arquivo/dados operacionais	Validação, persistência e registro de logs	Histórico de importação e atualização das bases analíticas
Mock em tempo real	Solicitação de simulação	Criação controlada de pedidos fictícios para demonstração	Atualização dos indicadores apresentados

5.1 Entradas principais do sistema

As principais entradas que tratamos no sistema são credenciais de autenticação, dados cadastrais, códigos de convite, filtros analíticos, arquivos de dados, registros de pedidos, campanhas e clientes. Padronizamos essas entradas por meio de formulários, parâmetros de consulta, payloads JSON e arquivos utilizados pelo processamento em Python.

5.2 Saídas principais do sistema

As principais saídas que entregamos são respostas JSON da API, tokens JWT, códigos de convite, e-mails transacionais, dashboards renderizados, indicadores financeiros, métricas de recorrência, segmentações de clientes, recomendações e mensagens de erro ou sucesso. No frontend, transformamos essas saídas em componentes visuais para facilitar a interpretação pelos usuários.

Indicador/KPI	Origem dos dados	Uso no projeto
Receita total	Pedidos e valores consolidados	Medir desempenho comercial geral
Total de pedidos	Tabela/arquivo de pedidos	Avaliar volume operacional
Ticket médio	Receita dividida pela quantidade de pedidos	Analisar valor médio de compra
Taxa de recorrência	Clientes com mais de um pedido	Avaliar fidelização e recompra
Ranking de empresas	Pedidos associados às empresas	Identificar melhores desempenhos
Recomendações	Regras analíticas e alertas Python	Sugerir ações de retenção, upsell e reativação

6. Nossas observações reflexivas

Durante o desenvolvimento, percebemos que a modelagem não deve ser tratada como uma etapa apenas teórica. Os diagramas UML nos ajudaram a visualizar responsabilidades, perfis de usuário, fluxos críticos e dependências entre camadas. Quando encontramos problemas de lentidão, autenticação, CORS, envio de e-mails e processamento do dashboard, a separação arquitetural facilitou a localização e a correção dos gargalos.

Um dos nossos aprendizados mais relevantes foi a importância de separar comunicação externa da resposta principal da API. Inicialmente, o envio de e-mail por SMTP fazia algumas rotas demorarem ou falharem por timeout. Ao migrarmos para envio assíncrono e, posteriormente, para a API HTTP da Brevo, reduzimos o impacto das chamadas externas e tornamos a aplicação mais estável.

Também percebemos que dashboards analíticos exigem cuidado com desempenho. O processamento em Python é útil para cálculos mais complexos, mas não deve ser executado de forma desnecessária em cada interação. Por isso, passamos a medir endpoints com logs de performance e a tratar cache e processamento em andamento de forma mais controlada.

Por fim, a entrega em ambiente hospedado reforçou para nós a necessidade de boas práticas de configuração. Precisamos alinhar variáveis de ambiente, CORS, banco em nuvem, URLs de frontend/backend, chaves de API e diferenças entre ambiente local e produção. Essa experiência tornou mais claro que design de software também envolve decisões operacionais, não apenas estrutura de código.

Desafio encontrado	Impacto no projeto	Aprendizado do grupo
Timeout no envio de e-mail	Convites e recuperação de senha demoravam ou falhavam	Integrações externas não devem bloquear a resposta da API
Processamento Python pesado	Dashboard podia demorar em filtros e recarregamentos	É necessário usar cache, logs e controle de execução
Diferença entre ambiente local e hospedado	Variáveis e URLs precisaram ser ajustadas	Configuração por ambiente deve estar documentada
Controle de perfis	Cada usuário precisava ver menus e dados diferentes	Permissões devem ser consideradas no design desde o início
Documentação e organização final	README e arquivos legados precisaram revisão	Projeto funcional também precisa ser compreensível para avaliadores

7. Conclusão

Ao aplicarmos design de software e UML no Dashboard de Indicadores Cannoli, conseguimos transformar uma solução inicialmente focada em visualização de dados em uma plataforma mais estruturada, com autenticação, perfis de acesso, integrações externas, processamento analítico e organização por camadas.

Os diagramas de casos de uso, classes e sequência documentam os principais comportamentos e estruturas que implementamos. Eles demonstram nossa compreensão da relação entre usuários, funcionalidades, entidades, banco de dados, backend, frontend e processamento analítico.

Como continuidade, recomendamos evoluir os testes automatizados, a documentação técnica, o monitoramento em produção e o refinamento da visão por empresa. Mesmo assim, consideramos que a arquitetura final já representa uma aplicação consistente dos conceitos de Engenharia de Software, Arquitetura de Sistemas e modelagem UML no contexto do Projeto Integrador.